🔍 SEO Audit API

A production-ready Node.js backend that performs comprehensive technical SEO audits on any website. It scrapes the page, analyzes on-page SEO elements, checks structured data, queries Google's PageSpeed Insights API, and returns a graded score with actionable fixes.

📋 Table of Contents

What It Does

The SEO Audit API analyzes a single URL and returns a comprehensive report covering:

Category	Checks
Meta Tags	Title length, meta description, Open Graph, Twitter Cards, canonical, robots, viewport
Headings	H1 count, H2/H3 presence, heading hierarchy (HTML5-aware)
Images	Alt text presence, lazy loading, width/height attributes, decorative image detection
Links	Internal/external counts, noopener security, generic anchor text, broken anchors
Technical	HTTPS, robots.txt, sitemap.xml, WWW redirect
Structured Data	JSON-LD schema detection with @graph support and validation
PageSpeed	Google Lighthouse scores (Performance, Accessibility, Best Practices, SEO) + Core Web Vitals

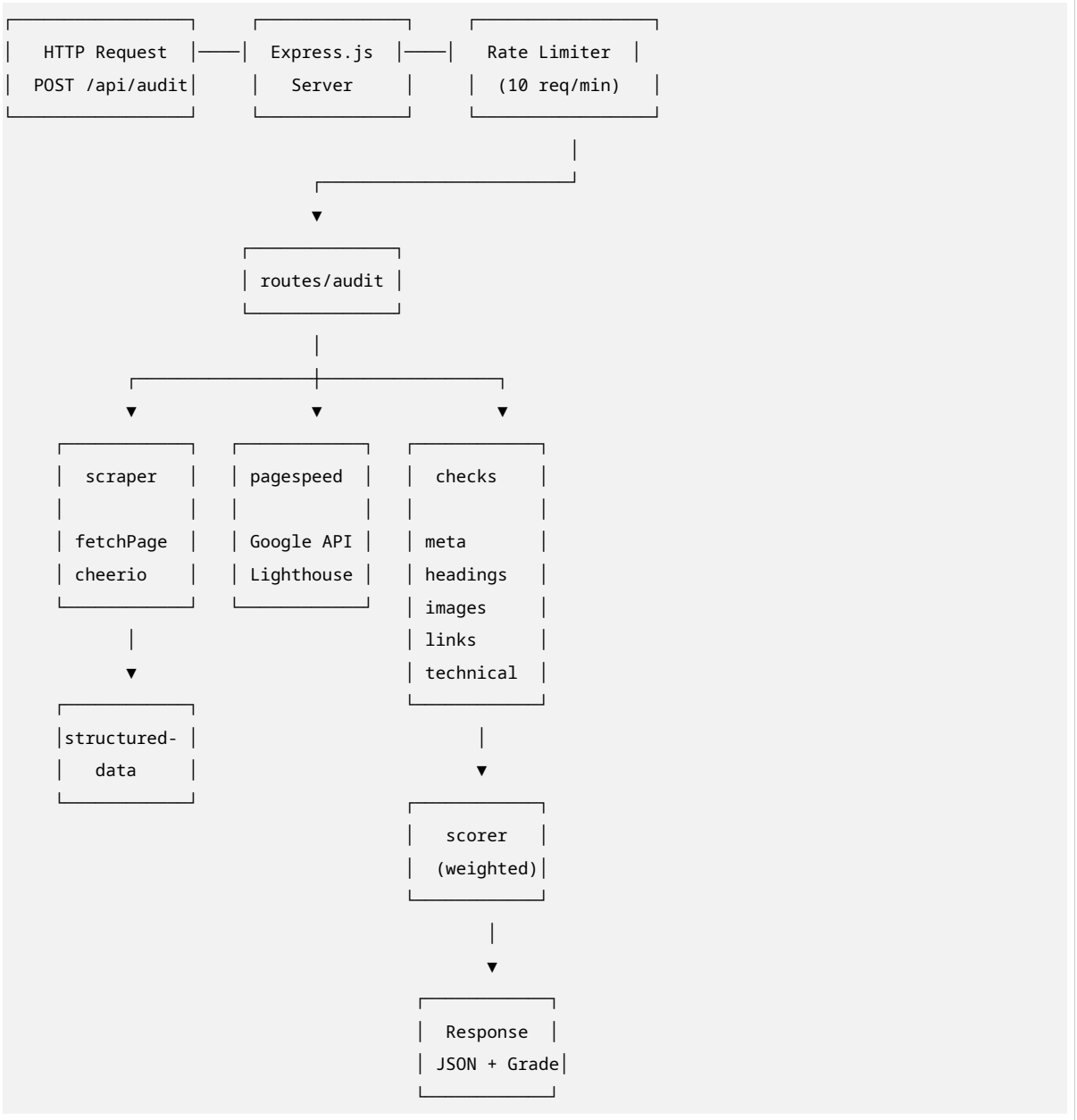
Key Features

- **Context-aware checks** — Distinguishes decorative images from missing alt text, JS-interactive links from broken links, and HTML5 sectioning from bad hierarchy
- **SPA detection** — Warns when auditing client-side rendered apps (React, Vue, Next.js)

- **Multi-path sitemap discovery** — Checks robots.txt, `/sitemap.xml`, `/sitemap_index.xml`, and more
- **Real PageSpeed data** — Integrates Google PageSpeed Insights API v5 with all Lighthouse categories
- **Proper error classification** — Returns appropriate HTTP status codes (400 DNS, 422 HTTP error, 504 timeout, etc.)
- **Automatic retries** — Exponential backoff for transient network failures

---

Architecture



File Structure

```
seo-audit-backend/
│
├── index.js                    # Express server entry point
├── package.json                # Dependencies and scripts
├── .env                        # Environment variables (not in git)
│
├── config/
│   └── index.js                # Constants, API keys, score weights
│
├── routes/
│   └── audit.js                # POST /api/audit endpoint + timeout wrappers
│
├── services/
│   ├── scraper.js              # Fetches page + SPA detection
│   ├── pagespeed.js            # Google PageSpeed Insights API integration
│   ├── structured-data.js      # JSON-LD schema parsing & validation
│   ├── scorer.js               # Weighted overall score calculation
│   └── robots.js               # robots.txt fetching & parsing
│
├── checks/
│   ├── index.js                # Orchestrates all checks
│   ├── meta.js                 # Title, description, OG, Twitter, canonical, robots, viewport
│   ├── headings.js             # H1/H2/H3 checks + HTML5-aware hierarchy
│   ├── images.js               # Alt text, lazy loading, dimensions
│   ├── links.js                # Internal/external, noopener, anchors
│   └── technical.js            # HTTPS, robots.txt, sitemap, WWW redirect
│
└── utils/
    ├── fetchPage.js            # HTTP fetch with retries & error handling
    ├── parseUrl.js             # URL normalization & internal link detection
    └── formatResult.js         # pass/warn/fail helper functions
```

File-by-File Reference

`index.js` — Server Entry Point

Sets up the Express application with security middleware, CORS, compression, rate limiting, and routes. Also handles 404s and global error handling.

Key middleware:

- `helmet()` — Security headers with CSP configuration
- `cors()` — Allows requests from `localhost:5173` and `localhost:3000`
- `compression()` — Gzip response compression
- `express-rate-limit` — 10 requests per minute per IP on `/api/*`

Endpoints:

- `POST /api/audit` — Run SEO audit
- `GET /health` — Health check with timestamp

`config/index.js` — Configuration

Centralizes all constants and environment variables:

Export	Description
`PORT`	Server port (default: 5000)
`PAGESPEED_API_KEY`	Google PageSpeed Insights API key
`USER_AGENT`	Bot user agent string for fetching
`IDEAL_TITLE_MIN/MAX`	50–60 characters
`IDEAL_META_DESC_MIN/MAX`	150–160 characters
`SCORE_WEIGHTS`	Category weights summing to 100

Weight distribution:

```
meta: 20, headings: 15, images: 15, links: 10,
structuredData: 15, technical: 10, pagespeed: 15
```

`routes/audit.js` — Audit Endpoint

`POST /api/audit`

Accepts `{ url: "https://example.com" }` and returns a complete audit report.

Flow:

1. Validates and normalizes the URL
2. Scrapes the page (with 25s timeout)
3. Runs all on-page checks
4. Queries PageSpeed API (with 70s timeout)
5. Extracts structured data
6. Calculates weighted overall score
7. Returns JSON response

Error handling: Classifies errors into proper HTTP status codes:

- `400` — Invalid URL, DNS not found, SSL certificate error
- `422` — Target site returned 4xx error
- `504` — Scrape or PageSpeed timeout
- `500` — Unexpected internal error

`services/scraper.js` — Page Scraper

Fetches the target URL and loads it into Cheerio for DOM parsing. Also detects if the page is a client-side rendered SPA.

SPA detection markers:

- `id="root"` or `id="app"` (React/Vue)
- `data-reactroot` (React)
- `__NEXT_DATA__` (Next.js)
- `window.__INITIAL_STATE__` (Vuex/Redux)

Returns: `{ $, html, meta: { finalUrl, statusCode, contentType, isSPA, title } }`

`services/pagespeed.js` — PageSpeed Integration

Queries Google's PageSpeed Insights API v5 for both mobile and desktop strategies. Requests all four Lighthouse categories explicitly.

Categories requested: `performance`, `accessibility`, `best-practices`, `seo`

Returns per device:

- `performanceScore`, `accessibilityScore`, `bestPracticesScore`, `seoScore`
- `coreWebVitals`: LCP, CLS, INP, FCP, TBT, TTI, SI

Fallback: If API key is missing/invalid, returns realistic fallback data with `isFallback: true` flag.

`services/structured-data.js` — Schema Parser

Finds and parses all `<script type="application/ld+json">` blocks. Handles:

- Single objects with `@type`
- `@graph` arrays (common in Yoast, RankMath)
- HTML comments and CDATA wrappers
- Basic validation for common schemas (Organization, Product, WebPage, BreadcrumbList)

Returns: `{ found, schemas[], count, invalidCount, invalidSchemas[], details[] }`

`services/scorer.js` — Score Calculator

Calculates category scores and overall weighted score.

Scoring logic:

- `pass` = 1.0 point
- `warning` = 0.7 points
- `info` = 0.8 points
- `fail` = 0 points

Grade scale:

Score	Grade
90–100	A
80–89	B
65–79	C
50–64	D
0–49	F

`services/robots.js` — robots.txt Parser

Fetches and parses `robots.txt` with user-agent awareness. Extracts `Disallow` paths and `Sitemap` references.

`checks/index.js` — Check Orchestrator

Loads the HTML into Cheerio and runs all check modules in parallel categories. Returns a structured object with `meta`, `headings`, `images`, `links`, and `technical` results.

`checks/meta.js` — Meta Tag Checks

Check	What It Validates
`checkTitle`	Length 50–60 chars, extracted from `<head>` only
`checkMetaDescription`	Length 150–160 chars
`checkOpenGraph`	og:title, og:description, og:image, og:url, og:type
`checkTwitterCard`	twitter:card, twitter:title, twitter:description, twitter:image
`checkCanonical`	Presence and URL match
`checkRobotsMeta`	No `noindex` / `nofollow` blocking
`checkViewport`	Presence and `width=device-width`

`checks/headings.js` — Heading Checks

Check	What It Validates
`checkH1`	Exactly one H1 (allows multiple in separate HTML5 sections)
`checkH2s`	Count and first 5 texts
`checkH3s`	Count
`checkHierarchy`	Heading sequence within same parent element only (not globally). Skips across different components are allowed.

`checks/images.js` — Image Checks

Check	What It Validates
`checkAltTexts`	Missing `alt` = **fail**. Empty `alt` with `role="presentation"` or `aria-hidden="true"` = **pass**. Small images (64px) and images inside labeled links/buttons are accepted.
`checkImageCount`	Total image count
`checkLazyLoading`	`loading="lazy"` or `data-src` pattern
`checkImageDimensions`	Missing `width`/`height` = **info** (may be CSS-handled). Large images (>200px) without dimensions = **warning**.

`checks/links.js` — Link Checks

Check	What It Validates
`checkInternalLinks`	Count of internal links
`checkExternalLinks`	Count of external links
`checkNoopener`	External links have `rel="noopener"`
`checkGenericAnchors`	Anchor text like “click here”, “read more”. Excludes links with `aria-label`, `title`, or inside cards with headings.
`checkBrokenAnchors`	Distinguishes: empty href (broken), `href="#"` with JS handlers (interactive), `href="#section"` (anchor link)

`checks/technical.js` — Technical Checks

Check	What It Validates
`checkHttps`	URL starts with `https://`
`checkRobotsTxt`	`robots.txt` exists and is parseable
`checkSitemap`	Tries multiple paths: `robots.txt` Sitemap directive, `/sitemap.xml`, `/sitemap_index.xml`, `/sitemap-index.xml`
`checkWwwRedirect`	Checks if `www` non-`www` redirect is properly configured

`utils/fetchPage.js` — HTTP Client

Wrapper around Axios with:

- Custom SEO bot user agent
- Gzip/deflate/br compression support
- 20s timeout, 5 max redirects
- **2 automatic retries** with exponential backoff (1s, 2s) for timeouts and 5xx errors
- Detailed error classification (DNS, SSL, timeout, connection refused)

`utils/parseUrl.js` — URL Utilities

Function	Purpose
`normalizeUrl`	Adds `https://` prefix, normalizes trailing slashes
`getDomain`	Extracts `protocol + hostname`
`isInternal`	Compares link domain to base URL (handles protocol-relative `//` links)
`isValidUrl`	Validates URL format

`utils/formatResult.js` — Result Helpers

Simple factory functions for consistent check results:

```
pass(message)   // { status: 'pass', message, fix: null }
warn(message, fix)  // { status: 'warning', message, fix }
fail(message, fix)  // { status: 'fail', message, fix }
```

API Reference

`POST /api/audit`

Request body:

```
{
  "url": "https://example.com"
}
```

Success response (200):

```
{
  "url": "https://example.com",
  "timestamp": "2026-06-27T06:58:17.808Z",
  "auditDuration": 48734,
  "overallScore": {
    "score": 88,
    "grade": "B",
    "breakdown": {
      "meta": 91,
      "headings": 83,
      "images": 83,
      "links": 74,
      "structuredData": 90,
      "technical": 100,
      "pagespeed": 92
    }
  },
```

```
  "pageSpeed": { "mobile": {...}, "desktop": {...} },
  "structuredData": { "found": true, "schemas": [...] },
  "checks": { "meta": {...}, "headings": {...}, "images": {...}, "links": {...}, "technical": {...} },
  "scrapeMeta": { "finalUrl", "statusCode", "isSPA", "title" },
  "warnings": []
}
```

Error response (example):

```
{
  "error": "Failed to perform audit",
  "type": "dns_error",
  "message": "Domain not found - check the URL spelling"
}
```

`GET /health`

Response:

```
{ "status": "ok", "timestamp": "2026-06-27T06:58:17.808Z" }
```

Environment Variables

Create a `.env` file in the project root:

```
PORT=5000
PAGESPEED_API_KEY=your_google_api_key_here
```

Getting a PageSpeed API key:

1. Go to Google Cloud Console
2. Create a project and enable the **PageSpeed Insights API**
3. Generate an API key under **Credentials**
4. Paste it into `.env`

> **Security note:** Never commit `.env` to git. Add it to `.gitignore`.

Installation

```
# Clone the repository
git clone <repo-url>
cd seo-audit-backend

# Install dependencies
npm install

# Create environment file
cp .env.example .env
# Edit .env and add your PageSpeed API key
```

```
# Start the server
npm start

# Or use nodemon for development
npm run dev
```

The server will start on `http://localhost:5000`.

Development

```
# Run with auto-restart on file changes
npm run dev

# Test the API
curl -X POST http://localhost:5000/api/audit \
  -H "Content-Type: application/json" \
  -d '{"url": "https://example.com"}'

# Health check
curl http://localhost:5000/health
```

Response Schema Details

`checks.meta`

Each check returns `{ status, message, fix }` where `status` is `pass`, `warning`, `fail`, or `info`.

`checks.headings`

- `h1: { status, message, fix }`
- `h2s: { status: 'info', count, texts[] }`
- `h3s: { status: 'info', count }`
- `hierarchy: { status, message, fix }`

`checks.images`

- `altTexts: { status, message, fix }`
- `count: { status: 'info', count }`
- `lazyLoading: { status, message, fix }`
- `dimensions: { status, message, fix }` (info for CSS-handled, warning for large missing)

`checks.links`

- `internal: { status: 'info', count }`
- `external: { status: 'info', count }`
- `noopener: { status, message, fix }`
- `genericAnchors: { status, message, fix }`

- brokenAnchors: { status, message, fix }

checks.technical

- https: { status, message, fix }
- robotsTxt: { status, message, sitemaps[] }
- sitemap: { status, message, fix }
- wwwRedirect: { status, message, fix }

pageSpeed


```
{
  "mobile": {
    "performanceScore": 59,
    "accessibilityScore": 82,
    "bestPracticesScore": 96,
    "seoScore": 92,
    "coreWebVitals": {
      "LCP": "11.2 s",
      "CLS": "0.073",
      "INP": "N/A",
      "FCP": "4.8 s",
      "TBT": "50 ms",
      "TTI": "11.3 s",
      "SI": "6.7 s"
    }
  },
  "desktop": { ... },
  "error": "",
  "isFallback": false
}
```

Tech Stack

Technology	Purpose
Node.js + Express	HTTP server and routing
Cheerio	Server-side DOM parsing (jQuery-like API)
Axios	HTTP client for fetching pages and APIs
Helmet	Security headers
CORS	Cross-origin request handling
express-rate-limit	API rate limiting
compression	Gzip response compression
dotenv	Environment variable management

License

ISC